CHAPTER OUTLINE

2.1 Physical Files and Logical Files

2.2 Opening Files

2.3 Closing Files

2.4 Reading and Writing
- 2.4.1 Read and Write Functions
- 2.4.2 A Program to Display the Contents of a File
- 2.4.3 Detecting End-of-File

2.5 Seeking
- 2.5.1 Seeking in C
- 2.5.2 Seeking in Pascal

2.6 Special Characters in Files

2.7 The UNIX Directory Structure

2.8 Physical and Logical Files in UNIX
- 2.8.1 Physical Devices as UNIX Files
- 2.8.2 The Console, the Keyboard, and Standard Error
- 2.8.3 I/O Redirection and Pipes

2.9 File-related Header Files

2.10 UNIX File System Commands

2.1 Physical Files and Logical Files

When we talk about a file on a disk or tape, we refer to a particular collection of bytes stored there. A file, when the word is used in this sense, physically exists. A disk drive might contain hundreds, even thousands, of these *physical files*.

From the standpoint of an application program, the notion of a file is different. To the program, a file is somewhat like a telephone line connected to a telephone network. The program can receive bytes through this phone line, or send bytes down it, but knows nothing about where these bytes actually come from or where they go. The program knows only about its own end of the phone line. Moreover, even though there may be thousands of physical files on a disk, a single program is usually limited to the use of only about 20 files.

The application program relies on the operating system to take care of the details of the telephone switching system, as illustrated in Fig. 2.1. It could be that bytes coming down the line into the program originate from an actual physical file, or they might come from the keyboard or some other input device. Similarly, the bytes that the program sends down the line might end up in a file, or they could appear on the terminal screen. Although the program often doesn't know where bytes are coming from or where they are going, it does know which line it is using. This line is usually

referred to as the *logical file* to distinguish this view from the *physical files* on the disk or tape.

Before the program can open a file for use, the operating system must receive instructions about making a hookup between a logical file (e.g., a phone line) and some physical file or device. When using operating systems such as IBM's OS/MVS, these instructions are provided through job control language (JCL). On minicomputers and microcomputers, more modern operating systems such as UNIX, MS-DOS, and VMS provide the instructions within the program. For example, in Turbo Pascal[†] the association between a logical file called *inp_file* and a physical file called *myfile.dat* is made with the following statement:

```
assign(inp_file,'myfile.dat')
```

This statement asks the operating system to find the physical file named *myfile.dat* and then to make the hookup by assigning a logical file (phone line) to it. The number identifying the particular phone line that is assigned is returned through the FILE variable *inp_file*, which is the file's *logical name*. This logical name is what we use to refer to the file inside the program. Again, the telephone analogy applies: My office phone is connected to six telephone lines. When I receive a call I get an intercom message such as, "You have a call on line three." The receptionist does not say, "You have a call from 918-123-4567." I need to have the call identified *logically*, not *physically*.

2.2 Opening Files

Once we have a logical file identifier hooked up to a physical file or device, we need to declare what we intend to do with the file. In general, we have two options: (1) open an *existing* file or (2) create a *new* file, deleting any existing contents in the physical file. Opening a file makes it ready for use by the program. We are positioned at the beginning of the file and are ready to start reading or writing. The file contents are not disturbed by the open statement. Creating a file also opens the file in the sense that it is ready for use after creation. Since a newly created file has no contents, writing is initially the only use that makes sense.

[†]Different Pascal compilers vary widely with regard to I/O procedures, since standard Pascal contains little in the way of I/O definition. Throughout this book we use the term *Pascal* when discussing features common to most Pascal implementations. When we refer to the features of a specific implementation, such as Turbo Pascal, we say so.

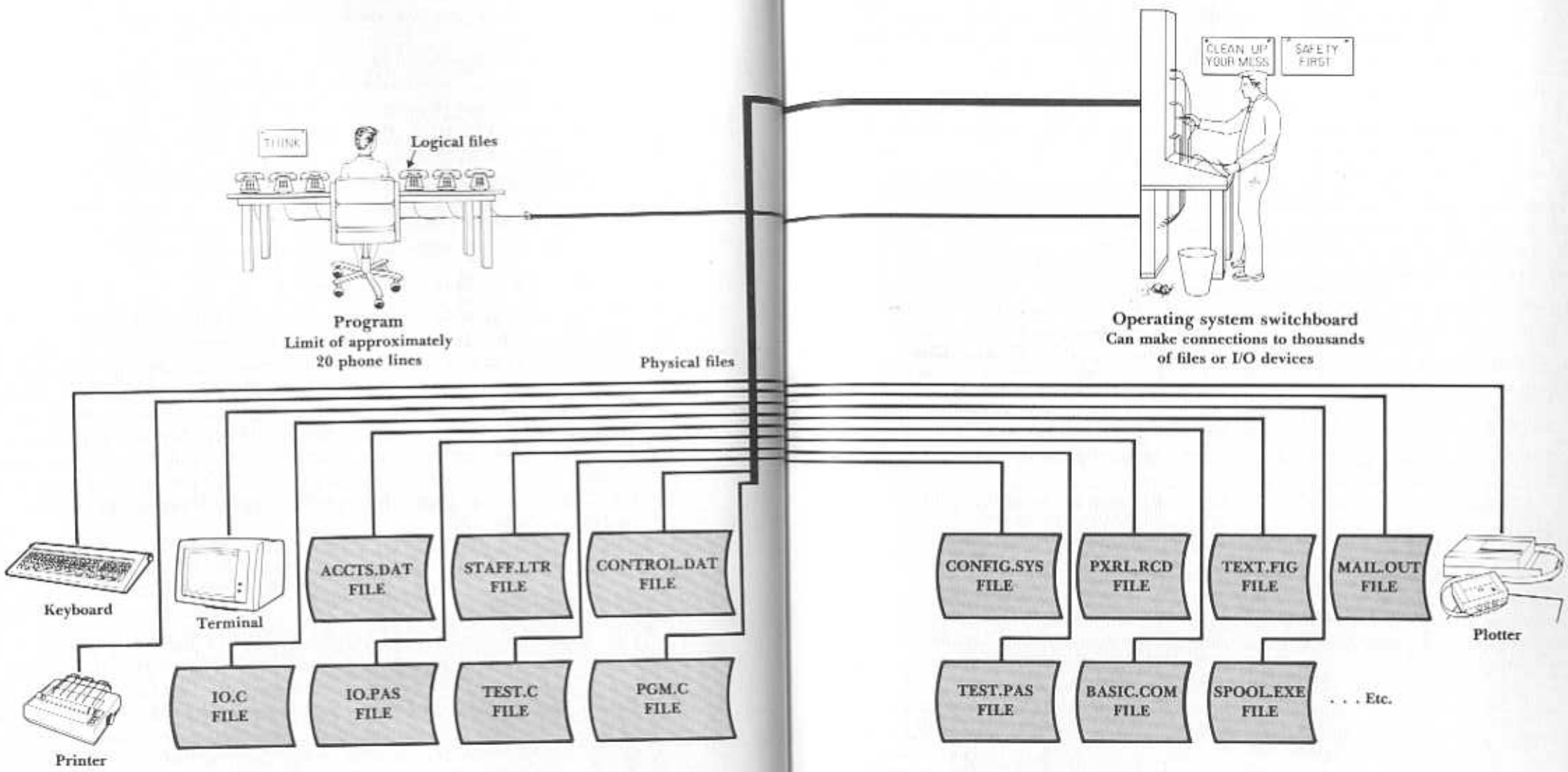


FIGURE 2.1 The program relies on the operating system to make connections between logical files and physical files and devices.

In Pascal the *reset( )* statement is used to open existing files and the *rewrite( )* statement is used to create new ones. For example, to open a file in Turbo Pascal we might use a sequence of statements such as:

```
assign(inp_file, 'myfile.dat');
reset (inp_file)
```

Note that we use the logical file name, not the physical one, in the *reset()* statement. To *create* a file in Turbo Pascal, the statements might read:

```
assign(out_file, 'myfile.dat');
rewrite (inp_file)
```

We can open an existing file or create a new one in C through the UNIX system function *open( )*. This function takes two required arguments and a third argument that is optional:

```
fd = open(filename, flags, [pmode]);
```

The return value *fd* and the arguments *filename*, *flags*, and *pmode* have the following meanings:

fd The *file descriptor*. Using our earlier analogy, this is the phone line (logical file identifier) used to refer to the file within the program. It is an integer. If there is an error in the attempt to open the file, this value is negative.

filename A character string containing the physical file name. (Later we discuss *pathnames* that include directory information about the file's location. This argument can be a pathname.)

flags The *flags* argument is an integer that controls the operation of the *open* function, determining whether it opens an existing file for reading or writing. It can also be used to indicate that you want to create a new file, or open an existing file but delete its contents. The value of *flags* is set by performing a bitwise OR of the following values, among others.†

O_APPEND	Append every write operation to the end of the file.
O_CREAT	Create and open a file for writing. This has no effect if the file already exists.
O_EXCL	Return an error if O_CREATE is specified and the file exists.
O_RDONLY	Open a file for reading only.
O_RDWR	Open a file for reading and writing.
O_TRUNC	If the file exists, truncate it to a length of zero, destroying its contents.
O_WRONLY	Open a file for writing only.

Some of these flags cannot be used in combination with one another. Consult your documentation for details, as well as for other options.

pmode If O_CREAT is specified, *pmode* is required. This integer argument specifies the *protection mode* for the file. In UNIX, the *pmode* is a three-digit octal number that indicates how the file can be used by the owner (first digit), by members of the owner's group (second digit), and by everyone else (third digit). The first bit of each octal digit indicates read permission, the second write permission, and the third execute permission. So, if *pmode* is the octal number 0751, the file's owner has read, write, and execute permission for the file; the owner's group would have read and execute permission; and everyone else has only execute permission:

```
                r w e   r w e   r w e
PMODE = 0751 =  1 1 1   1 0 1   0 0 1
                owner   group   world
```

†These values are defined in an "include" file packaged with your UNIX system or C compiler. The name of the include file is often *fcntl.h* or *file.h*, but it can vary from system to system.

Given this description of the *open( )* function, we can develop some examples to show how it can be used to open and create files in C. The following function call opens an existing file for reading and writing, or creates a new one if necessary. If the file exists it is opened without change; reading or writing would start at the file's first byte.

```
fd = open(filename, O_RDWR | O_CREAT, 0751);
```

The following call creates a new file for reading and writing. If there is already a file with the name specified in *filename*, its contents are truncated.

```
fd = open(filename, O_RDWR | O_CREAT | O_TRUNC, 0751);
```

Finally, here is a call that will create a new file only if there is not already a file with the name specified in *filename*. If a file with this name exists, it is not opened and the function returns a negative value to indicate an error.

```
fd = open(filename, O_RDWR | O_CREAT | O_EXCL,  0751);
```

File protection is tied more to the host operating system than to a specific language. For example, implementations of Pascal running on systems that support file protection, such as VAX/VMS, often include extensions to standard Pascal that let you associate a protection status with a file when you create it.

2.3 Closing Files

In terms of our telephone line analogy, closing a file is like hanging up the phone. When you hang up the phone, the phone line is available for taking or placing another call; when you close a file, the logical file name or file descriptor is available for use with another file. Closing a file that has been used for output also ensures that everything has been written to the file. As you will learn in a later chapter, it is more efficient to move data to and from secondary storage in blocks than it is to move data one byte at a time.

Consequently, the operating system does not immediately send off the bytes we write, but saves them up in a buffer for transfer as a block of data. Closing a file makes sure that the buffer for that file has been flushed of data and that everything we have written has actually been sent to the file.

Files are usually closed automatically by the operating system when a program terminates normally. Consequently, the explicit use of a CLOSE statement within a program is needed only as protection against data loss in the event of program interruption and to free up logical filenames for reuse. Some languages, including Standard Pascal, do not even provide a CLOSE statement. However, explicit file closing is possible in the C language, VAX Pascal, PL/I, and most other languages used for serious file processing work.

Now that you know how to connect and disconnect programs to and from physical files and how to open the files, you are ready to start sending and receiving data.

2.4 Reading and Writing

Reading and *writing* are fundamental to file processing; they are the actions that make file processing an *input/output* (I/O) operation. The actual form of the read and write statements used in different languages varies. Some languages provide very high-level access to reading and writing and automatically take care of details for the programmer. Other languages provide access at a much lower level. Our use of Pascal and C allows us to explore some of these differences.[†]

2.4.1 Read and Write Functions

We begin here with reading and writing at a relatively low level. It is useful to have a kind of systems-level understanding of what happens when we send and receive information to and from a file.

A low-level read call requires three pieces of information, expressed here as arguments to a generic READ( ) function.

```
READ(Source_file,Destination_addr,Size)
```

`Source_file`	The READ( ) call must know from where it is to read. We specify the source by logical file name (phone line) through which data is received. (Remember, before we do any reading we must have already opened the file, so the connection between a logical file and a specific physical file or device already exists.)
`Destination_addr`	READ( ) must know where to place the information it reads from the input file. In this generic function we specify the destination by giving the first address of the memory block where we want to store the data.
`Size`	Finally, READ( ) must know how much information to bring in from the file. Here the argument is supplied as a byte count.

A *WRITE* statement is similar; the only difference is that the data moves in the other direction:

```
WRITE(Destination_file,Source_addr,Size)
```

`Destination_file`	The logical file name we use for sending the data.
`Source_addr`	WRITE( ) must know where to find the information that it will send. We provide this specification as the first address of the memory block where the data is stored.
`Size`	The number of bytes to be written must be supplied.

2.4.2 A Program to Display the Contents of a File

Let's do some reading and writing to see how these functions are used. This first simple file processing program, which we call LIST, opens a file for input and reads it, character by character, sending each character to the screen after it is read from the file. LIST includes the following steps:

1. Display a prompt for the name of the input file.
2. Read the user's response from the keyboard into a variable called *filename*.
3. Open the file for input.
4. While there are still characters to be read from the input file,
   a. read a character from the file and
   b. write the character to the terminal screen.
5. Close the input file.

Figures 2.2 and 2.3 are, respectively, C and Pascal language implementations of this program. It is instructive to look at the differences between these implementations.

[†]To accentuate these differences and provide a look at I/O operations at something closer to a systems level, we use the *read( )* and *write( )* system calls in C rather than higher-level functions such as *fgetc( )*, *fgets( )*, and so on.

```
/* list.c--program to read characters from a file and write them
**          to the terminal screen
*/
#include <stdio.h>
#include <fcntl.h>

main( )
{

        char c;
        int  fd;    /* file descriptor */
        char filename[20];

        printf("Enter the name of the file: ");      /* Step 1 */
        gets(filename);                              /* Step 2 */
        fd =open(filename, O_RDONLY);                /* Step 3 */

        while (read(fd, &c, 1) != 0)                 /* Step 4a */
              write(STDOUT, &c, 1);                  /* Step 4b */

        close(fd);                                   /* Step 5 */
```

FIGURE 2.2 The LIST program in C.

Steps 1 and 2 of the program involve writing and reading, but in each of the implementations this is accomplished through the usual functions for handling the screen and keyboard. Step 4a, where we read from the input file, is the first instance of actual file I/O. Note that the *read( )* call in the C language parallels the low-level, generic READ( ) statement we described earlier; in truth, we used the *read( )* system call in C as the model for our low-level READ( ). The function's first argument gives the file descriptor (C's version of a logical file name) as the *source* for the input, the second argument gives the *address* of a character variable used as the *destination* for the data, and the third argument specifies that only one byte will be read.

The arguments for the Pascal *read( )* call communicate the same information at a higher level. Once again, the first argument is the logical file name for the input source. The second argument gives the *name* of a character variable used as a destination; given the name, Pascal can find the address. Because of Pascal's strong emphasis on variable types, the third argument of the generic READ( ) function is not required. Pascal assumes that since we are reading data into a variable of type *char*, we must want to read only one byte.

After a character is read, we write it out to the screen in Step 4b. Once again the differences between C and Pascal indicate the range of approaches to I/O used in different languages. Everything must be stated explicitly in

the C *write( )* call. Using the special, assigned file descriptor of *STDOUT* to identify the terminal screen as the destination for our writing,

```
write( STDOUT, &c, 1);
```

means: "Write to the screen the contents from memory starting at the address &c. Write only one byte." Beginning C programmers should pay special attention to the use of the & symbol in the *write( )* call here; this particular C call, as a very low-level call, requires that the programmer provide the starting *address* in RAM of the bytes to be transferred.

STDOUT, which stands for "standard output," is an integer value defined in the file *stdio.h*, which has been included at the top of the program. The actual value of *STDOUT* that is set in *stdio.h* is, by convention, always 1. The concept of standard output and its counterpart "standard input" are covered later in the section "Physical and Logical Files in UNIX."

FIGURE 2.3 The LIST program in Pascal.

```
PROGRAM list (INPUT, OUTPUT);

{ reads input from a file and writes it to the terminal screen }

VAR

      c        : char;
      infile   : file of char;                  { logical file nam

      filename : packed array [1..20] of char; { physical file na

BEGIN {main}

      write('Enter the name of the file: ');     { Step 1 }
      readln(filename);                          { Step 2 }
      reset(infile, filename);                   { Step 3 }
      while not (eof(infile)) DO

      BEGIN

            read(infile,c);                      { Step 4a }
            write(c)                             { Step 4b }

      END;

      close(infile)                              { Step 5 }

END.
```

Again the Pascal code operates at a higher level.† When no logical file name is specified in a *write( )* statement, Pascal assumes that we are writing to the terminal screen. Since the variable *c* is of type *char*, Pascal assumes we are writing a single byte. The statement becomes simply

```
write(c)
```

As in the *read( )* statement, Pascal takes care of finding the *address* of the bytes; the programmer need specify only the name of the variable *c* that is associated with that address.

2.4.3 Detecting End-of-File

The programs in Figs. 2.2 and 2.3 have to know when to end the *while* loop and stop reading characters. Pascal and C signal the end-of-file condition differently, illustrating two of the most commonly used approaches to end-of-file detection.

Pascal supplies a Boolean function, *eof( )*, which can be used to test for end-of-file. As we read from a file, the operating system keeps track of our location in the file with a *read/write pointer*. This is necessary so when the next byte is read, the system knows where to get it. The *eof( )* function queries the system to see whether the read/write pointer has moved past the last element in the file. If it has, *eof( )* returns *true*; otherwise it returns *false*. As Fig. 2.3 illustrates, we use the *eof( )* call before trying to read the next byte. For an empty file, *eof( )* immediately returns *true* and no bytes are read.

In the C language, the *read( )* call returns the number of bytes read. If *read( )* returns a value of zero, then the program has reached the end of the file. So, rather than using an *eof( )* function, we construct the *while* loop to run as long as the *read( )* call finds something to read.

2.5 Seeking

In the preceding sample programs we read through the file *sequentially*, reading one byte after another until we reach the end of the file. Every time a byte is read, the operating system moves the read/write pointer ahead, and we are ready to read the next byte.

†This is not to say that C does not have similar high-level functions. In fact, the standard C library provides a panoply of higher-level I/O functions, including *putc( )*, which functions for characters exactly like the Pascal *write( )* shown here. We have chosen to emphasize the use of the lower-level C functions mainly for pedagogical reasons. They provide opportunities for us to understand more fully the way file I/O works.

Sometimes we want to read or write without taking the time to go through every byte sequentially. Perhaps we know that the next piece of information we need is 10,000 bytes away, and so we want to jump there to begin reading. Or perhaps we need to jump to the end of the file so we can add new information there. To satisfy these needs we must be able to control the movement of the read/write pointer.

The action of moving directly to a certain position in a file is often called *seeking*. A *seek* requires at least two pieces of information, expressed here as arguments to the generic pseudocode function SEEK( ):

```
SEEK(Source_file,Offset)
```

`Source_file`	The logical file name in which the seek will occur.
`Offset`	The number of positions in the file the pointer is to be moved from the start of the file.

Now, if we want to move directly from the origin to the 373rd position in a file called *data*, we don't have to move sequentially through the first 372 positions first. Instead, we can say

```
SEEK( data, 373 )
```

2.5.1 Seeking in C

One of the features of UNIX that has been incorporated into many implementations of the C language is the ability to view a file as a potentially very large *array of bytes* that just happens to be kept on secondary storage. In an array of bytes in RAM, we can move to any particular byte through the use of a subscript. The C language seek function, called *lseek( )*, provides a similar capability for files. It lets us set the read/write pointer to any byte in a file.

The *lseek( )* function has the following form:

```
pos = lseek(fd, byte_offset, origin)
```

where the variables have the following meanings:

pos	A long integer value returned by *lseek( )* equal to the position (in bytes) of the read/write pointer after it has been moved.
fd	The file descriptor of the file to which the *lseek( )* is to be applied.
byte_offset	The number of bytes to move from some *origin* in the file. The byte offset must be specified as a long integer, hence the name *lseek* for long seek. When appropriate, the *byte_offset* can be negative.

origin A value that specifies the starting position from which the *byte_offset* is to be taken. The *origin* can have the value 0, 1, or 2:†

0—*lseek( )* from the beginning of the file;

1—*lseek( )* from the current position;

2—*lseek( )* from the end of the file.

The following program fragment shows how you could use *lseek( )* to move to a position that is 373 bytes into a file.

```
long pos,lseek(int fd, long offset, int origin);
int fd;
.
.
pos=lseek(fd, 373L, 0);
```

2.5.2 Seeking in Pascal

The view of a file as presented in Pascal differs from the C view in at least two important respects:

- In C a file is a sequence of bytes, so addressing within the file is on a byte-by-byte basis. When we seek to a position, we express the address in terms of bytes. In Pascal a file is a sequence of "records" of some particular type. A record can be a simple scalar such as a character or integer, or it may be a more complex structure. Addressing within a file in Pascal is in terms of these records. For example, if a file is made up of 100-byte records, and we want to refer to the fourth record, we would do so in Pascal simply by referencing record number 4. In C, where the view is solely and always in terms of bytes, we would have to address the fourth record as byte address 400.
- Standard Pascal actually does not provide for seeking. The model for I/O for standard Pascal is magnetic tape, which must be read sequentially. In standard Pascal, adding data to the end of a file requires reading the entire file from beginning to end, writing out the data from the input file to a second, output file, and then adding the new data to the end of the output file. However, many implementations of Pascal such as VAX Pascal and Turbo Pascal have extended the standard and do support seeking.

†Although the values 0, 1, and 2 are almost always used here, they are not guaranteed to work for all C implementations. Consult your documentation.

There is an extension to Pascal proposed by the Joint ANSI/IEEE Pascal Standards Committee (1984) that may be included in the Pascal standard in the future. It includes the following procedures and functions that permit seeking:

SeekWrite(f,n) A procedure that positions the file *f* on the element with index *n* and places the file in write mode, so the selected and following elements may be modified.

SeekRead(f,n) A procedure that positions the file *f* on the element with index *n* and places the file in read mode, so the selected and following elements may be examined. If SeekRead( ) attempts to position beyond the end of the file, then the file is positioned at the end of the file.

Position(f) A function that returns the index value representing the position of the current file element.

EndPosition(f) A function that returns the index value representing the position of the last file element.

Many Pascal implementations, recognizing the need to provide seeking capabilities, had already implemented seeking functions before these proposals were set forth. Consequently, the mechanisms for handling seeking vary widely among implementations.

2.6 Special Characters in Files

As you create the file structures described in this text, you may encounter some difficulty with extra, unexpected characters that turn up in your files, with characters that disappear, and with numeric counts that are inserted into your files. Here are some examples of the kinds of things you might encounter:

- On many small computers you may find that a Control-Z (ASCII value of 26) is appended at the end of your files. Some applications use this to indicate end-of-file even if you have not placed it there. This is most likely to happen on MS-DOS systems.
- Some systems adopt a convention of indicating end-of-line in a text file† as a pair of characters consisting of a carriage return (CR: ASCII value of 13) and a line feed (LF: ASCII value of 10). Sometimes I/O

†When we use the term *text file* in this text, we are referring to a file consisting entirely of characters from a specific standard character set, such as ASCII or EBCDIC. Unless otherwise specified, the ASCII character set will be assumed. Appendix A contains a table that describes the ASCII character set.

procedures written for such systems automatically expand single CR characters or LF characters into CR-LF pairs. This unrequested addition of characters can cause a great deal of difficulty. Again, you are most likely to encounter this phenomenon on MS-DOS systems.

- Users of larger systems, such as VMS, may find that they have just the opposite problem. Certain file formats under VMS *remove* carriage return characters from your file without asking you, replacing them with a *count* of the characters in what the system has perceived as a line of text.

These are just a few examples of the kinds of uninvited modifications that record management systems or I/O support packages might make to your files. You will find that they are usually associated with the concepts of a line of text or end of a file. In general, these modifications to your files are an attempt to make your life easier by doing things for you automatically. This might, in fact, work out for users who want to do nothing more than store some text in a file. Unfortunately, however, programmers building sophisticated file structures must sometimes spend a lot of time finding ways to disable this automatic assistance so they can have complete control over what they are building. Forewarned is forearmed; readers who encounter these kinds of difficulties as they build the file structures described in this text can take some comfort from the knowledge that the experience they gain in disabling automatic assistance will serve them well, over and over, in the future.

2.7 The UNIX Directory Structure

No matter what computer system you have, even if it is a small PC, chances are there are hundreds or even thousands of files you have access to. To provide convenient access to such large numbers of files, your computer has some method for organizing its files. In UNIX this is called the *filesystem*.

The UNIX filesystem is a tree-structured organization of *directories*, with the *root* of the tree signified by the character '/'. All directories, including the root, can contain two kinds of files: regular files with programs and data, and directories (Fig. 2.4). Since devices such as tape drives are also treated like files in UNIX, directories can also contain references to devices, as shown in the *dev* directory in Fig. 2.4. The file name stored in a UNIX directory corresponds to what we call its physical name.

Since every file in a UNIX system is part of the filesystem that begins with root, any file can be uniquely identified by giving its *absolute pathname*. For instance, the true, unambiguous name of the file "addr" in Fig. 2.4 is

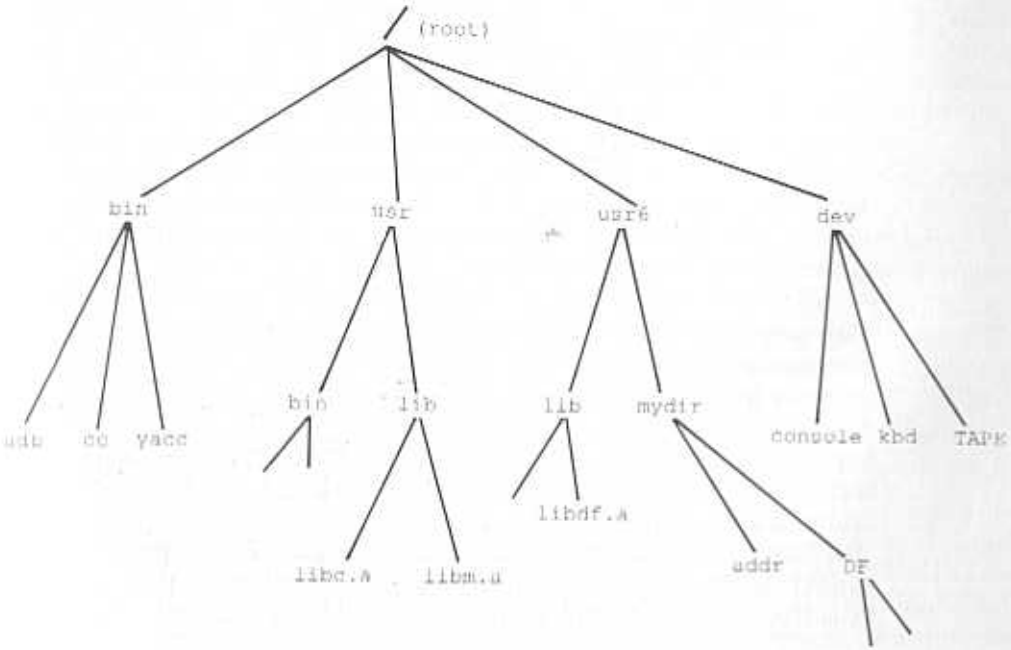


FIGURE 2.4 Sample UNIX directory structure.

/usr6/mydir/addr. (Note that the '/' is used both to indicate the root directory and to separate directory names from the file name.)

When you issue commands to a UNIX system, you do so within some directory, which is called your *current directory*. A pathname for a file that does not begin with a '/' describes the location of a file relative to the current directory. Hence, if your current directory in Fig. 2.4 is mydir, *addr* uniquely identifies the file */usr6/mydir/addr*.

The special filename "." stands for the current directory, and ".." stands for the parent of the current directory. Hence, if your current directory is */usr6/mydir/DF*, "../addr" refers to the file */usr6/mydir/addr*.

2.8 Physical and Logical Files in UNIX

2.8.1 Physical Devices as UNIX Files

One of the most powerful ideas in UNIX is reflected in its notion of what a file is. In UNIX, a file is a sequence of bytes, without any implication of how or where the bytes are stored or where they originate. This simple

conceptual view of a file makes it possible in UNIX to do with a very few operations what might require many times as many operations on a different operating system. For example, it is easy to think of a magnetic disk as the source of a file, because we are used to the idea of storing such things on disks. But in UNIX, devices like the keyboard and the console are also files—in Fig. 2.4, */dev/kbd* and */dev/console*, respectively. The keyboard produces a sequence of bytes that are sent to the computer when keys are pressed; the console accepts a sequence of bytes and displays their corresponding symbols on a screen.

How can we say that the UNIX concept of a file is simple when it allows so many different physical things to be called files? Doesn't this make the situation more complicated, not simpler? The trick in UNIX is that no matter what physical representation a file may take, the logical view of a UNIX file is the same. In its simplest form, a UNIX file is represented logically by an integer—the file descriptor. This integer is an index to an array of more complete information about the file. A keyboard, a disk file, and a magnetic tape are all represented by integers. Once the integer that describes a file is identified, a program can access that file. If it knows the logical name of a file, a program can access that file without knowing whether the file comes from a disk, a tape, or a telephone.

2.8.2 The Console, the Keyboard, and Standard Error

We see an example of the duality between devices and files in the LIST program in Fig. 2.2:

```
printf("Enter the name of the file: "); /* Step 1 */
gets(filename);                         /* Step 2 */
fd =open(filename, O_RDONLY);           /* Step 3 */

while (read(fd, &c, 1) > 0)             /* Step 4a */
write(STDOUT, &c, 1);                   /* Step 4b */
```

The logical file *fd* is some small integer value returned by the *open( )* call. We assign this integer to the variable *fd* in Step 3. In Step 4b, we use the integer *STDOUT*, defined as 1 earlier in the program, to identify the console as the file to be written to.

There are two other file descriptors that are special in UNIX: The keyboard is called *STDIN (standard input)* and the error file is called *STDERR (standard error)*. Hence, *STDIN* is the keyboard on your terminal. The statement

```
read(STDIN, &c, 1);
```


reads a single character from your terminal. *STDERR* is an error file which, like *STDOUT*, is usually just your console. When your compiler detects an error, it generally writes the error message to this file, which means normally that the error message turns up on your screen. As with *STDIN*, the values *STDIN* and *STDERR* are usually defined in *stdio.h*.

Steps 1 and 2 of the LIST program also involve reading and writing from *STDIN* or *STDOUT*. Since an enormous amount of I/O involves these devices, most programming languages have special functions to perform console input and output—in LIST, the C functions *printf* and *gets* are used. Ultimately, however, *printf* and *gets* send their output through *STDOUT* and *STDIN*, respectively. But these statements hide important elements of the I/O process. For our purposes, the second set of read and write statements is more interesting and instructive.

2.8.3 I/O Redirection and Pipes

Suppose you would like to change the LIST program so it writes its output to a regular file, rather than to *STDOUT*. Or suppose you wanted to use the output of LIST as input to another program. Because it is common to want to do both of these, UNIX provides convenient shortcuts for switching between standard I/O (*STDIN* and *STDOUT*) and regular file I/O. These shortcuts are called *I/O redirection* and *pipes*.[†]

I/O redirection lets you specify at execution time alternate files for input or output. The notations for input and output redirection are

```
< file          (redirect STDIN to "file")
> file          (redirect STDOUT to "file")
```

For example, if the executable LIST program is called "list," we redirect the output from *STDOUT* to a file called "myfile" by entering the line

```
list > myfile
```

What if, instead of storing the output from the list program in a file, you wanted to use it immediately in another program to sort the results? UNIX pipes let you do this. The notation for a UNIX pipe is '|'. Hence,

```
program1 | program2
```

[†]Strictly speaking, I/O redirection and pipes are part of a UNIX shell, which is the command interpreter that sits on top of the core UNIX operating system, the kernel. For the purpose of this discussion, this distinction is not important.

means take any *STDOUT* output from program1 and use it in place of any *STDIN* input to program2. Since UNIX has a special program called *sort*, which takes its input from *STDIN*, you can sort the output from the list program, without using an intermediate file, by entering

```
list | sort
```

Since *sort* writes its output to *STDOUT*, the sorted listing appears on your terminal screen unless you use additional pipes or redirection to send it elsewhere.

2.9 File-related Header Files

UNIX, like all operating systems, has special names and values that you must use when performing file operations. For example, some C functions return a special value indicating end-of-file (EOF) when you try to read beyond the end of a file.

Recall the flags that you use in an *open( )* call to indicate whether you want read-only, write-only, or read/write access. Unless we know just where to look, it is often not easy to find where these values are defined. UNIX handles the problem by putting such definitions in special header files such as /usr/include, which can be found in special directories.

Three header files relevant to the material in this chapter are *stdio.h*, *fcntl.h*, and *file.h*. EOF, for instance, is defined on many UNIX systems in */usr/include/stdio.h*, as are the file pointers STDIN, STDOUT, and STDERR. And the flags O_RDONLY, O_WRONLY, and O_RDWR can usually be found in */usr/include/sys/file.h* or possibly one of the files that it includes.

It would be instructive for you to browse through these files, as well as others that pique your curiosity.

2.10 UNIX Filesystem Commands

UNIX provides many commands for manipulating files. We list a few that are relevant to the material in this chapter. Most of them have many options, but the simplest uses of most should be obvious. Consult a UNIX manual for more information on how to use them.

`cat` *filenames*	Print the contents of the named text files.
`tail` *filename*	Print the last 10 lines of the text file.

`cp` *file1 file2*	Copy *file1* to *file2*.
`mv` *file1 file2*	Move (rename) *file1* to *file2*.
`rm` *filenames*	Remove (delete) the named files.
`chmod` *mode filename*	Change the protection mode on the named files.
`ls`	List the contents of the directory.
`mkdir` *name*	Create a directory with the given name.
`rmdir` *name*	Remove the named directory.

SUMMARY

This chapter introduces the fundamental operations of file systems: OPEN( ), CREATE( ), CLOSE( ), READ( ), WRITE( ), and SEEK( ). Each of these operations involves the creation or use of a link between a *physical file* stored on a secondary device and a *logical file* that represents a program's more abstract view of the same file. When the program describes an operation using the *logical file name*, the equivalent physical operation gets performed on the corresponding physical file.

The six operations appear in programming languages in many different forms. Sometimes they are built-in commands, sometimes they are functions, and sometimes they are direct calls to an operating system. Not all languages provide the user with all six operations. The operation SEEK( ), for instance, is not available in standard Pascal.

Before we can use a physical file, we must link it to a logical file. In some programming environments we do this with a statement (e.g., *assign* in Turbo Pascal) or with instructions outside of the program (e.g., job control language [JCL] instructions). In other languages the link between the physical file and a logical file is made with OPEN( ) or CREATE( ).

The operations CREATE( ) and OPEN( ) make files ready for reading or writing. CREATE( ) causes a new physical file to be created. OPEN( ) operates on an already existing physical file, usually setting the read/write pointer to the beginning of the file. The CLOSE( ) operation breaks the link between a logical file and its corresponding physical file. It also makes sure that the file buffer is flushed so everything that was written is actually sent to the file.

The I/O operations READ( ) and WRITE( ), when viewed at a low, systems level, require three items of information:

- The *logical name* of the file to be read from or written to;

- An *address* of a memory area to be used for the "inside of the computer" part of the exchange; and
- An indication of *how much data* is to be read or written.

These three fundamental elements of the exchange are illustrated in Fig. 2.5.

READ( ) and WRITE( ) are sufficient for moving sequentially through a file to any desired position, but this form of access is often very inefficient. Some languages provide seek operations that let a program move directly to a certain position in a file. C provides direct access by means of the *lseek( )* operation. The *lseek( )* operation lets us view a file as a kind of large array, giving us a great deal of freedom in deciding how to organize a file. Standard Pascal does not support direct file access, but many dialects of Pascal do.

One other useful file operation involves knowing when the end of a file has been reached. End-of-file detection is handled in different ways by different languages.

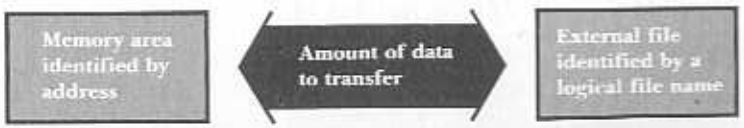
Much effort goes into shielding programmers from having to deal with the physical characteristics of files, but inevitably there are little details about the physical organization of files that programmers must know. When we try to have our program operate on files at a very low level (as we do a great deal in this text), we must be on the lookout for little surprises inserted in our file by the operating system or applications.

The UNIX file system, called the *filesystem*, organizes files in a tree structure, with all files and subdirectories expressable by their *pathnames*. It is possible to navigate around the filesystem as you work with UNIX files.

UNIX views both physical devices and traditional disk files as files, so, for example, a keyboard (*STDIN*), a console (*STDOUT*), and a tape drive all are considered files. This simple conceptual view of files makes it possible in UNIX to do with a very few operations what might require many times the operations on a different operating system.

I/O redirection and *pipes* are convenient shortcuts provided in UNIX for transferring file data between files and *standard I/O*. *Header files* in UNIX, such at *stdio.h*, contain special names and values that you must use when

FIGURE 2.5 The exchange between memory and external device.



performing file operations. It is important to be aware of the most common of these in use on your system.

The following section lists a sampling of UNIX commands for manipulating files.

KEY TERMS

Access mode. Type of file access allowed. The variety of access modes permitted varies from operating system to operating system.

Buffering. When input or output is saved up rather than sent off to its destination immediately, we say that it is *buffered*. In later chapters, we find that we can dramatically improve the performance of programs that read and write data if we buffer the I/O.

Byte offset. The distance, measured in bytes, from the beginning of the file. The very first byte in the file has an offset of 0, the second byte has an offset of 1, and so on.

CLOSE( ). A function or system call that breaks the link between a logical file name and the corresponding physical file name.

CREATE( ). A function or system call that causes a file to be created on secondary storage and may also bind a logical name to the file's physical name—see OPEN( ). A call to CREATE( ) also results in the generation of information used by the system to manage the file, such as time of creation, physical location, and access privileges for anticipated users of the file.

End-of-file (EOF). An indicator within a file that the end of the file has occurred, a function that tells if the end of a file has been encountered (e.g., *eof( )* in Pascal), or a system-specific value that is returned by file-processing functions indicating that the end of a file has been encountered in the process of carrying out the function (e.g., EOF in UNIX).

File descriptor. A small, non-negative integer value returned by a UNIX *open( )* or *creat( )* call that is used as a logical name for the file in later UNIX system calls.

Filesystem. The name used in UNIX to describe a collection of files and directories organized into a tree-structured hierarchy.

Header file. A file in a UNIX environment that contains definitions and declarations commonly shared among many other files and applications. In C, header files are included in other files by means of the "#include" statement (see Fig. 2.2). The header files *stdio.h*, *file.h*,

and *fcntl.h* described in this chapter contain important declarations and definitions used in file processing.

I/O redirection. The redirection of a stream of input or output from its normal place. For instance, the operator '>' can be used to redirect to a file output that would normally be sent to the console.

Logical file. The file as seen by the program. The use of logical files allows a program to describe operations to be performed on a file without knowing what actual physical file will be used. The program may then be used to process any one of a number of different files that share the same structure.

OPEN( ). A function or system call that makes a file ready for use. It may also bind a logical file name to a physical file. Its arguments include the logical file name and the physical file name and may also include information on how the file is expected to be accessed.

Pathname. A character string that describes the location of a file or directory. If the pathname starts with a '/', then it gives the *absolute pathname*—the complete path from the root directory to the file. Otherwise it gives the *relative pathname*—the path relative to the current working directory.

Physical file. A file that actually exists on secondary storage. It is the file as known by the computer operating system and that appears in its file directory.

Pipe. A UNIX operator specified by the symbol '|' that carries data from one process to another. The originating process specifies that the data is to go to *STDOUT*, and the receiving process expects the data from *STDIN*. For example, to send the standard output from a program *makedata* to the standard input of a program called *usedata*, use the command "makedata | usedata".

Protection mode. An indication of how a file can be accessed by various classes of users. In UNIX, the protection mode is a three-digit octal number that indicates how the file can be read, written to, and executed by the owner, by members of the owner's group, and by everyone else.

READ( ). A function or system call used to obtain input from a file or device. When viewed at the lowest level, it requires three arguments: (1) a Source_file logical name corresponding to an open file; (2) the Destination_address for the bytes that are to be read; and (3) the Size or amount of data to be read.

SEEK( ). A function or system call that sets the read/write pointer to a specified position in the file. Languages that provide seeking functions allow programs to access specific elements of a file *directly*, rather than having to read through a file from the beginning (*sequentially*) each time a specific item is desired. In C, the *lseek( )* system call provides this capability. Standard Pascal does not have a seeking capability, but many nonstandard dialects of Pascal do.

Standard I/O. The source and destination conventionally used for input and output. In UNIX, there are three types of standard I/O: *standard input (STDIN)*, *standard output (STDOUT)*, and *STDERR (standard error)*. By default *STDIN* is the keyboard, and *STDOUT* and *STDERR* are the console screen. I/O redirection and pipes provide ways to override these defaults.

WRITE( ). A function or system call used to provide output capabilities. When viewed at the lowest level, it requires three arguments: (1) a Destination_file name corresponding to an open file; (2) the Source_address of the bytes that are to be written; and (3) the Size or amount of the data to be written.

EXERCISES

1. Look up operations equivalent to *OPEN( ), CLOSE( ), CREATE( ), READ( ), WRITE( )*, and *SEEK* in other high-level languages, such as PL/I, COBOL, and Fortran. Compare them with the C or Pascal versions.

2. If you use C:
   a) Make a list of the different ways to perform the file operations CREATE( ), OPEN( ), CLOSE( ), READ( ), and WRITE( ). Why is there more than one way to do each operation?
   b) How would you use *lseek( )* to find the current position in a file?
   c) Show how to change the permissions on a file *myfile* so the owner has read and write permissions, group members have execute permission, and others have no permission.
   d) What is the difference between *pmode* and O_RDWR? What *pmodes* and O_RDWR are available on your system?
   e) In some typical C environments, such as UNIX and MS-DOS, all of the following represent ways to move data from one place to another:

```
scanf( )   fgetc( ) read( ) cat (or type)
fscanf( )  gets( )  <       main (argc,argv)
getc( )    fgets( ) |
```

   Describe as many of these as you can, and indicate how they might be useful. Which belong to the C language, and which belong to the operating system?

3. If you use Pascal:
   a) What ways are provided in your version of Pascal to perform the file operations CREATE( ), OPEN( ), CLOSE( ), READ( ), and WRITE( )? If there is more than one way to do a certain operation, tell why. If an operation is missing, how are its functions carried out?
   b) Implement a SEEK( ) function in your Pascal, if it does not already have one.

4. A couple of years ago a company we know of bought a new COBOL compiler. One difference between the new compiler and the old one was that the new compiler did not automatically close files when execution of a program terminated, whereas the old compiler did. What sorts of problems did this cause when some of the old software was executed after having been recompiled with the new compiler?

5. Look at the two LIST programs in the text. Each has a *while* loop. In Pascal, the sequence of steps in the loop is test, read, write. In C, it is read, test, write. Why the difference? What would happen in Pascal if we used the loop construction used for C? What would happen in C if we used the Pascal loop construction?

6. In Fig. 2.4:
   a. Give the full pathname for a file in directory *DF*.
   b. Suppose your current directory is *bin*. Show how to copy the file *libdf.a* to the directory *DF* without changing your current directory.

7. What is the difference between *STDOUT* and *STDERR*? Find how to direct error messages from a compilation on your system to *STDERR*.

8. Look up the UNIX command *wc*. Execute the following in a UNIX environment, and explain why it gives the number of files in the directory.

```
ls | wc -w
```

9. Find *stdio.h* on your system, and find what value is used to indicate end-of-file. Also examine *file.h* or *fcntl.h* and describe in general what its contents are for.

Programming Exercises

10. Make the LIST program we provide in this chapter work with your compiler on your operating system.

11. Write a program to create a file and store a string in it. Write another program to open the file and read the string.

12. Try setting the *protection mode* on a file to read-only, then opening the file with an access mode of read/write. What happens?

13. Implement the UNIX command *tail -n*, where *n* is the number of lines from the end of the file to be copied to *STDOUT*.

14. Change the program LIST so it reads from the *STDIN*, rather than a file, and writes to a file, rather than the *STDOUT*. Show how to execute the new version of the program in a UNIX environment, given that the input is actually in a file called *instuff*. (You can also do this in most MS-DOS environments.)

15. Write a program to read a series of names, one per line, from standard input, and write out those names spelled in reverse order to standard output. Use I/O redirection and pipes to do the following:
   a. Input a series of names that are typed in from the keyboard and write them out, reversed, to a file called *file1*.
   b. Read the names in from *file1*; then write them out, re-reversed, to a file called *file2*.
   c. Read the names in from *file2*, reverse them again, and then sort the resulting list of reversed words using *sort*.

FURTHER READINGS

Introductory textbooks on C and Pascal tend to treat the fundamental file operations only briefly, if at all. This is particularly true with regard to C, since there are higher-level standard I/O functions in C, such as the read operations *fgets( )* and *fgetc( )*. Some books on C and/or UNIX that do provide treatment of the fundamental file operations are Bourne (1984), Kernighan and Pike (1984), and Kernighan and Ritchie (1978, 1988). These books also provide discussions of higher-level I/O functions that we omitted from our text.

As for UNIX specifically, as of this writing there are two dominant flavors of UNIX: UNIX System V from AT&T, the originators of UNIX, and 4.3BSD (Berkeley Software Distribution) UNIX from the University of California at Berkeley. The two versions are close enough that learning about either will give you a good understanding of UNIX generally. However, as you begin to use UNIX, you will need reference material on the specific version that you are using. There are many accessible texts on both versions, including Morgan and McGilton (1987) on System V, and Wang (1988) on 4.3BSD. Less readable but absolutely essential to a serious UNIX user is the 4.3BSD *UNIX Programmers Reference Manual* (U.C. Berkeley, 1986) or the *System V Interface Definition* (AT&T, 1986).

For Pascal, these operations vary so greatly from one implementation to another that it is probably best to consult user's manuals and literature relating to

your specific implementation. Cooper (1983) covers the ISO standard Pascal, as well as some extensions. Jensen and Wirth (1974) is the definition of Pascal on which all others are based. Wirth (1975) discusses some difficulties with standard Pascal and file operations in the section, "An Important Concept and a Persistent Source of Problems: Files."

3 Secondary Storage and System Software

CHAPTER OBJECTIVES

- Describe the organization of typical disk drives, including basic units of organization and their relationships.
- Identify and describe the factors affecting disk access time, and describe methods for estimating access times and space requirements.
- Describe magnetic tapes, identify some tape applications, and investigate the implications of block size on space requirements and transmission speeds.
- Identify fundamental differences between media and criteria that can be used to match the right medium to an application.
- Describe in general terms the events that occur when data is transmitted between a program and a secondary storage device.
- Introduce concepts and techniques of buffer management.
- Illustrate many of the concepts introduced in the chapter, especially system software concepts, in the context of UNIX.